Checking for Convergence

When you run gradient descent, it's important to monitor its progress to ensure it's actually working and approaching the minimum cost.

The Learning Curve 📈

The most common way to check for convergence is to plot a **learning curve**.

- **What it is:** A graph of the cost function, $J(w, b)$, versus the **number of iterations** of gradient descent.
    - **Horizontal axis:** Number of iterations.
    - **Vertical axis:** Cost J.
- **How to Interpret It:**
    - **If gradient descent is working correctly:** The cost `J` should **decrease** after every single iteration. If the cost ever *increases*, it's a sign that your learning rate `α` is likely too large, or there might be a bug in your code.
    - **When has it converged?** The algorithm has **converged** when the learning curve flattens out. This means that additional iterations are no longer significantly decreasing the cost.
- **Important Note:** The number of iterations needed for convergence can vary greatly between different problems (from dozens to hundreds of thousands). The learning curve is the best way to visualize this for your specific application.

Automatic Convergence Test

An alternative to visual inspection is an **automatic convergence test**.

- **How it works:** You choose a very small number, called **epsilon** (`ε`), for example, `0.001`. If the cost `J` decreases by less than epsilon in a single iteration, you can declare that the algorithm has converged.
- **Practicality:** While this method exists, it can be difficult to choose the right threshold for epsilon. Visually inspecting the learning curve is often a more reliable and insightful approach.